Reed-Solomon 코드

개요

Reed-Solomon(RS) 코드는 zkSNARK 프로토콜에서 다항식의 저차성을 검증하는 핵심 도구입니다. 오류 정정 능력과 근접성 테스트를 통해 증명의 무결성을 보장합니다.

Reed-Solomon 코드의 정의

기본 개념

- **메시지**: k개의 심볼로 구성된 데이터
- **코드워드**: n개의 심볼로 확장된 인코딩 결과 (n > k)
- **최소 거리**: d = n - k + 1 (최대 (d-1)/2개의 오류 정정 가능)

수학적 정의

메시지 (m_0, m_1, ..., m_{k-1})에 대해:

1. 다항식 구성: $P(x) = m_0 + m_1x + ... + m_{k-1}x^{k-1}$
2. 평가점 선택: $\alpha_0, \alpha_1, ..., \alpha_{n-1} \in F$
3. 코드워드 생성: $(P(\alpha_0), P(\alpha_1), ..., P(\alpha_{n-1}))$

구현

```
template<typename FieldT>
class ReedSolomonCode {
```

```
private:
size_t message_length; // k
size_t codeword_length; // n
vector<FieldT> evaluation_domain; // α₀, α₁ , ..., α_{n-1}
public:
ReedSolomonCode(size_t  k, size_t  n, const vector<FieldT>&  domain)
: message_length(k), codeword_length(n), evaluation_domain(domain) {
assert(n > k);
assert(domain.size() == n);
}
// 인코딩: 메시지 → 코드워드
vector<FieldT> encode(const vector<FieldT>&  message) const {
assert(message.size() == message_length);
vector<FieldT> codeword(codeword_length);
// 각 평가점에서 다항식 값 계산
for (size_t i = 0; i < codeword_length; i++) {
codeword[i] = horner_evaluate(message, evaluation_domain[i]);
}
return codeword;
}
// 디코딩: 코드워드 → 메시지 (오류 정정 포함)
vector<FieldT> decode(const vector<FieldT>&  received) const {
assert(received.size() == codeword_length);
```

```
// 베를캄프-웰치 알고리즘 또는 유클리드 알고리즘 사용
return berlekamp_welch_decode(received);
}
private:
vector<FieldT> berlekamp_welch_decode(
const vector<FieldT>& received
) const {
// 오류 위치 다항식과 오류 평가 다항식 계산
// 복잡한 구현이므로 개념적 설명만 제공
// 1. 신드롬 계산
// 2. 오류 위치 다항식 계산
// 3. 오류 값 계산
// 4. 원본 메시지 복구
return vector<FieldT>(); // 실제 구현 필요
}
};
```

근접성 테스트 (Proximity Test)

기본 아이디어

주어진 워드가 어떤 저차 다항식의 인코딩에 “가까운지” 확인합니다.

직접 테스트 방법

```
template<typename FieldT>
class DirectProximityTest {
private:
size_t degree_bound;
vector<FieldT> evaluation_domain;
public:
// 저차성 테스트
bool  test_low_degree(
const vector<FieldT>&  word,
size_t  num_queries
) const {
// 무작위 쿼리 생성
vector<size_t> query_positions =
generate_random_queries(num_queries);
for (size_t pos : query_positions) {
if (!check_local_consistency(word, pos)) {
return false;
}
}
return true;
}
private:
```

```
bool check_local_consistency(
const vector<FieldT>& word,
size_t position
) const {
// 주변 점들을 이용한 지역적 일관성 검사
vector<size_t> nearby_positions =
get_nearby_positions(position);
// 라그랑주 보간으로 예상 값 계산
vector<FieldT> x_coords, y_coords;
for (size_t pos : nearby_positions) {
if (pos != position) {
x_coords.push_back(evaluation_domain[pos]);
y_coords.push_back(word[pos]);
}
}
LagrangeInterpolation<FieldT> interpolator(x_coords, y_coords);
FieldT expected = interpolator.evaluate(evaluation_domain[position]);
return word[position] == expected;
}
};
```

FRI 기반 테스트

```
template<typename FieldT>
class FRIProximityTest {
private:
vector<size_t> domain_sizes; // 각 라운드의 도메인 크기
vector<FieldT> folding_challenges; // 폴딩 챌린지
public:
// FRI 프로토콜 실행
bool  run_fri_test(
const vector<FieldT>&  initial_polynomial,
size_t  degree_bound
) {
vector<FieldT> current_poly = initial_polynomial;
// 폴딩 라운드들
for (size_t round = 0; round < domain_sizes.size() - 1; round++) {
// 폴딩 수행
current_poly = fold_polynomial(
current_poly,
folding_challenges[round]
);
// 커밋 (실제로는 머클 트리 루트)
commit_polynomial(current_poly);
}
```

```
// 최종 다항식이 상수인지 확인

return  is_constant_polynomial(current_poly);

}

private:

vector<FieldT> fold_polynomial(

const vector<FieldT>&  poly,

const FieldT&  challenge

) const {

size_t n = poly.size();

vector<FieldT> folded(n / 2);

for (size_t i = 0; i < n / 2; i++) {

// f(x) = f₀(x²) + x·f₁ (x²)에서

// f(challenge) = f₀(challenge²) + challenge·f₁ (challenge²)

folded[i] = poly[2*i] + challenge * poly[2*i + 1];

}

return folded;

}

};
```

코드워드 도메인과 시스템 도메인

도메인 분리의 필요성

- **시스템 도메인**: 실제 계산이 수행되는 도메인

- **코드워드 도메인**: RS 인코딩을 위한 확장된 도메인
- **분리 이유**: 보안성과 효율성의 균형

도메인 설정

```
template<typename FieldT>
class DomainConfiguration {
private:
vector<FieldT> systematic_domain; // H
vector<FieldT> codeword_domain; // L (H ⊂ L)
size_t expansion_factor; // |L| / |H|
public:
DomainConfiguration(
size_t  systematic_size,
size_t  expansion_factor
) : expansion_factor(expansion_factor) {
// 시스템 도메인 생성 (곱셈 부분군)
FieldT generator = FieldT::multiplicative_generator();
FieldT omega = generator.pow(
(FieldT::mod - 1) / systematic_size
);
systematic_domain.resize(systematic_size);
systematic_domain[0] = FieldT::one();
for (size_t i = 1; i < systematic_size; i++) {
```

```
systematic_domain[i] = systematic_domain[i-1] * omega;
}
// 코드워드 도메인 생성
size_t codeword_size = systematic_size * expansion_factor;
FieldT codeword_omega = generator.pow(
(FieldT::mod - 1) / codeword_size
);
codeword_domain.resize(codeword_size);
codeword_domain[0] = FieldT::one();
for (size_t i = 1; i < codeword_size; i++) {
codeword_domain[i] = codeword_domain[i-1] * codeword_omega;
}
}
// 시스템 도메인에서 코드워드 도메인으로 확장
vector<FieldT> extend_to_codeword_domain(
const vector<FieldT>&  systematic_values
) const {
assert(systematic_values.size() == systematic_domain.size());
// 라그랑주 보간으로 다항식 복구
LagrangeInterpolation<FieldT> interpolator(
systematic_domain,
systematic_values
);
```

```
// 코드워드 도메인에서 평가
vector<FieldT> codeword_values(codeword_domain.size());
for (size_t i = 0; i < codeword_domain.size(); i++) {
codeword_values[i] = interpolator.evaluate(codeword_domain[i]);
}
return codeword_values;
}
};
```

최적화 기법

배치 인코딩

```
template<typename FieldT>
class BatchRSEncoder {
private:
OptimizedNTT<FieldT> ntt;
public:
// 여러 메시지를 동시에 인코딩
vector<vector<FieldT>> batch_encode(
const vector<vector<FieldT>>&  messages
) const {
vector<vector<FieldT>> codewords(messages.size());
```

```
#pragma omp parallel for
for (size_t i = 0; i < messages.size(); i++) {
codewords[i] = encode_single(messages[i]);
}
return codewords;
}
private:
vector<FieldT> encode_single(const vector<FieldT>& message) const {
vector<FieldT> padded_message = message;
// NTT 크기로 패딩
size_t ntt_size = next_power_of_two(message.size() * expansion_factor);
padded_message.resize(ntt_size);
// INTT로 계수 → 평가값 변환
ntt.ntt(padded_message, true);
return padded_message;
}
};
```

메모리 효율적 디코딩

```
template<typename FieldT>
class StreamingRSDecoder {
public:
```

```
// 스트리밍 방식으로 대용량 코드워드 처리
vector<FieldT> streaming_decode(
function<FieldT(size_t)> codeword_accessor,
size_t  codeword_length
) const {
// 청크 단위로 처리하여 메모리 사용량 최소화
const  size_t chunk_size = 1024;
vector<FieldT> syndrome(message_length);
for (size_t chunk_start = 0;
chunk_start < codeword_length;
chunk_start += chunk_size) {
size_t chunk_end = min(chunk_start + chunk_size, codeword_length);
// 청크별 신드롬 계산
for (size_t i = chunk_start; i < chunk_end; i++) {
FieldT value = codeword_accessor(i);
update_syndrome(syndrome, i, value);
}
}
// 신드롬으로부터 오류 정정
return  correct_errors_from_syndrome(syndrome);
}
};
```

보안성 분석

최소 거리와 오류 정정 능력

- **최소 거리**: d = n - k + 1
- **오류 검출**: d - 1개까지 검출 가능
- **오류 정정**: ⌊(d-1)/2⌋개까지 정정 가능

근접성 테스트의 보안성

- **사운드니스**: 저차가 아닌 다항식을 높은 확률로 거부
- **완전성**: 저차 다항식을 항상 수락
- **쿼리 복잡도**: O(log n)개의 쿼리로 충분한 보안성

다음 단계

이제 수학적 기초를 마쳤으므로, R1CS 관계 섹션에서 제약 시스템의 구조를 학습하겠습니다.

← 이전: 다항식 이론 | 다음: R1CS 관계 →